QR Code Recognition Control Action (PTZ)

1. Experiment Objective

The camera (pan-tilt) scans QR codes and recognizes encoded commands (forward/back/left/right/turnleft/turnright/stop) to control the chassis to perform the corresponding actions. The PTZ initializes to a forward-facing angle and does not participate in tracking.

2. Experiment Principle

1. Key Terms

Term	Description
Visual Control	The camera captures an image → the target content is recognized → mapped to chassis control commands, forming a "perceive → decide → execute" loop
QR Command Mapping	7 normalized commands (forward/back/left/right/turnleft/turnright/stop) → predefined linear velocity + angular velocity combinations
pyzbar / OpenCV QR	Dual-backend detection: pyzbar is based on the zbar library (node default), OpenCV uses the built-in QRCodeDetector (launch default)
Joy Override	Subscribes to the `/JoyState` topic; when the joystick is active, the node automatically stops publishing `/cmd_vel` and yields control
Command Hold Timeout (hold_timeout)	After the QR code is lost, the last valid command continues to be executed for `qr_command_hold_timeout_sec`, then the car stops
PTZ Initialization	Publishes the initial pan-tilt angle 20 times at 0.2 s intervals at startup (horizontal 0°, pitch -45°), ensuring the camera faces straight ahead
QoS	best_effort (image + cmd_vel), reliable (JoyState/beep)
TF Coordinate Frame	odom → base_footprint → base_link

2. Technical Architecture

The node uses a **dual-timer architecture** (image processing at 30 Hz + chassis control at 20 Hz). It shares the same node source code as the No-PTZ version, with the only differences being the added pan-tilt initialization and `/cmd_ptz_angle` publishing:

```
/camera/image_raw (BEST_EFFORT)
  → qr_control_ptz_node
    ├── Image processing timer: image conversion → flip/rotate → pyzbar/OpenCV QR detection → state update
    ├── Chassis control timer: QR command mapping → /cmd_vel (BEST_EFFORT) → firmware execution
    └── Pan-tilt initialization: publishes the initial angle 20 times at startup → /cmd_ptz_angle
/JoyState (Bool) → joy override → zero velocity + buzzer off
/beep (UInt16) ← buzzer feedback for unknown commands
```

3. Core Topics Quick Reference

Firmware Uplink

Topic	Message Type	QoS	Description
`/camera/image_raw`	`sensor_msgs/Image`	best_effort	Raw image frame from the ESP32 camera

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
`/cmd_vel`	`geometry_msgs/Twist`	best_effort + keep_last(1)	`linear.x`	-0.30~0.30 m/s	`+=` forward, `-=` backward
			`angular.z`	-1.50~1.50 rad/s	`+=` turn left, `-=` turn right
`/cmd_ptz_angle`	`microros_v2_msgs/PtzAngle`	reliable	`horizontal_deg`	-90~90°	Pan-tilt horizontal angle
			`pitch_deg`	-90~0°	Pan-tilt pitch angle

ROS2 Internal Topics

Topic	Message Type	QoS	Source	Description
`/JoyState`	`std_msgs/Bool`	reliable	Joystick node	true when the joystick is active; the node stops the car automatically

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☑
No-PTZ version	❌
Required peripherals	ESP32 camera (pan-tilt) + PTZ servo

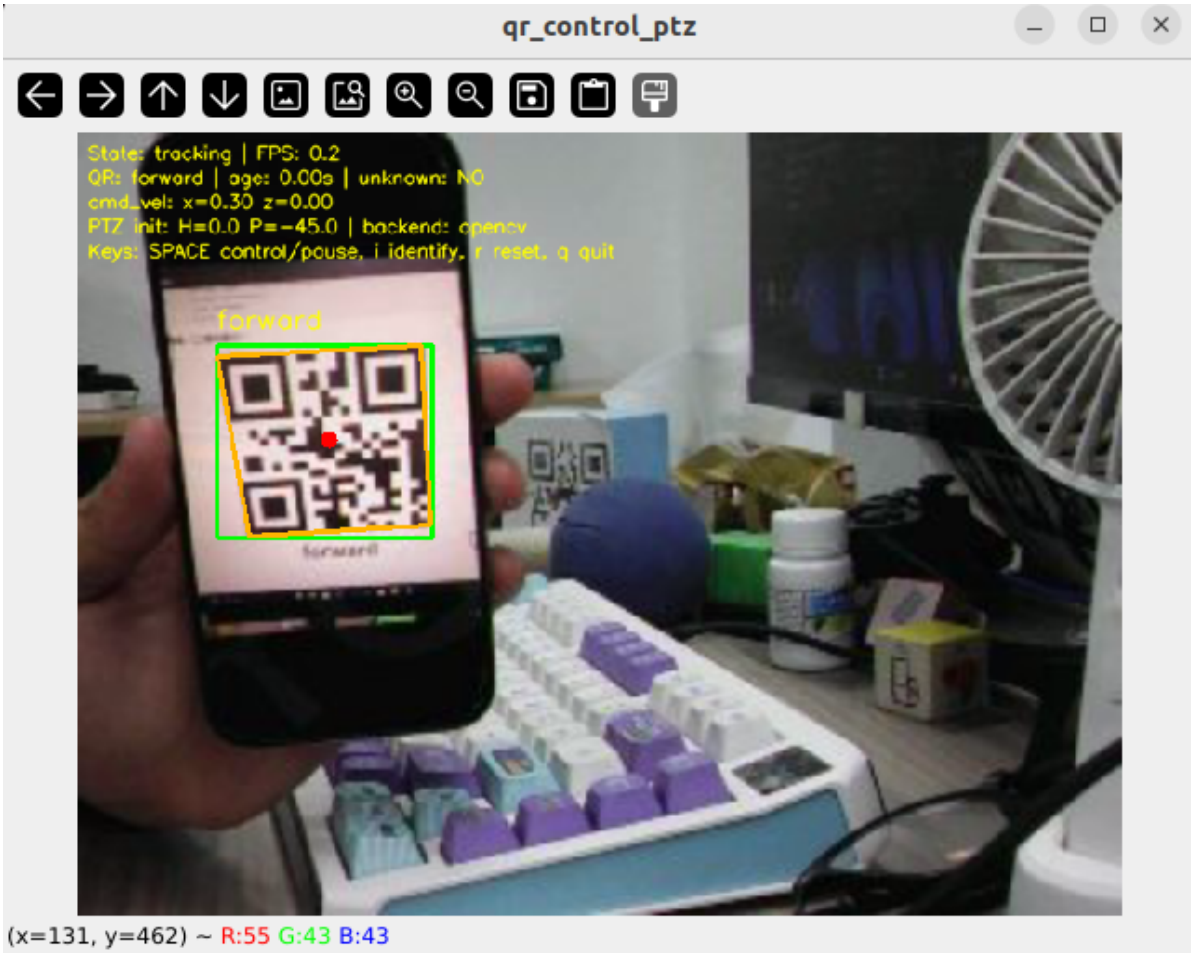
2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info     | UDPv4AgentLinux.cpp | init                    | running...              | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start QR Code Control (Terminal 2)

```
ros2 launch microros_v2_ptz_visual_control_pkg qr_control_ptz.launch.py
```



4. Operation Instructions

Key	Function	Description
Space	Control switch	Enable/pause QR control; publishes zero velocity when paused
i / I	Recognition mode	Only recognize and display, without decoding to control the chassis
r / R	Reset state	Clear the current QR code cache and stop control
q / Q / ESC	Exit	Stop the car + `os._exit(0)`

QR Command Mapping:

Normalized Command	Linear Velocity (m/s)	Angular Velocity (rad/s)	Behavior
`forward`	0.30	0	Move forward
`back`	-0.30	0	Move backward
`left`	0	1.00	Turn left in place
`right`	0	-1.00	Turn right in place
`turnleft`	0.30	1.50	Move forward + turn left
`turnright`	0.30	-1.50	Move forward + turn right
`stop`	0	0	Stop

5. How to Stop

Press `Ctrl+C` to stop Terminal 2 → Terminal 1 in order.

4. Experiment Observations

- **No QR code + control enabled**: The frame shows "QR: None", `cmd_vel` is zero velocity, state `lost`
- **QR code = forward**: The car drives straight ahead at 0.30 m/s
- **QR code = turnleft**: The car moves forward at 0.30 m/s + turns left at 1.50 rad/s
- **QR code lost for > 1.5 s**: The car stops automatically, state `lost`
- **Unknown QR code**: With `unknown_command_stop=true`, the car stops + optional buzzer
- **Joystick active**: The car stops immediately and resumes after release
- **Multiple codes**: Selects the largest code using the `largest` strategy

5. Program Analysis

> Source code paths:

> - Launch: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/qr_control_ptz.launch.py`
> - Node: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/qr_control_ptz_node.py`

1. Core Node Analysis

`qr_control_ptz_node` shares the QR detection and control logic with the No-PTZ version. PTZ-specific features:

- **Pan-tilt initialization**: Publishes the initial pan-tilt angle 20 times at 0.2 s intervals at startup (`initial_horizontal_deg`=0°, `initial_pitch_deg`=-45°), ensuring the servo reaches the target position
- `/cmd_ptz_angle` **publisher**: Publishes the pan-tilt angle command (horizontal + pitch)
- **On-screen overlay**: Includes the initial pan-tilt angle information

Constructor — pan-tilt publisher and initialization:

```
# Source file: ~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/qr_control_ptz_node.py

def __init__(self) -> None:
    super().__init__(DEFAULT_NODE_NAME)
    # ...(image/QR state initialization same as No-PTZ version)...

    # PTZ-specific: pan-tilt angle publisher
    self.cmd_ptz_angle_publisher = self.create_publisher(
        PtzAngle, "/cmd_ptz_angle", 10
    )
    # Pan-tilt initialization parameters
    self.declare_parameter("initial_horizontal_deg", 0.0)
    self.declare_parameter("initial_pitch_deg", -45.0)
    self.declare_parameter("initial_publish_count", 20)

    # Publish the initial pan-tilt angle after startup (20 times, 0.2 s interval)
    self._publish_initial_ptz_angle()
```

2. Key Parameters

> Parameter definition files:
>
> - Launch: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/qr_control_ptz.launch.py`
> - Node: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/qr_control_ptz_node.py`

Parameter	Type	Default Value	Description
`forward_linear_speed`	float	`0.30` m/s	Linear speed for the forward command
`back_linear_speed`	float	`-0.30` m/s	Linear speed for the back command
`rotate_angular_speed`	float	`1.00` rad/s	Angular speed for left/right rotation
`turn_linear_speed`	float	`0.30` m/s	Linear speed for the turn commands
`turn_angular_speed`	float	`1.50` rad/s	Angular speed for the turn commands
`qr_detector_backend`	str	`opencv`	Detection backend: pyzbar / opencv / auto
`qr_command_hold_timeout_sec`	float	`1.5` s	Command hold time after the code is lost
`control_rate_hz`	float	`20.0` Hz	Chassis control frequency
`image_processing_rate_hz`	float	`30.0` Hz	Image processing frequency
`auto_start_control`	bool	`false`	Start control automatically after launch
`stop_when_qr_lost`	bool	`true`	Stop the car after the code-lost timeout
`enable_joy_override`	bool	`true`	Joystick override switch
`initial_horizontal_deg`	float	`0.0` °	Initial pan-tilt horizontal angle
`initial_pitch_deg`	float	`-45.0` °	Initial pan-tilt pitch angle

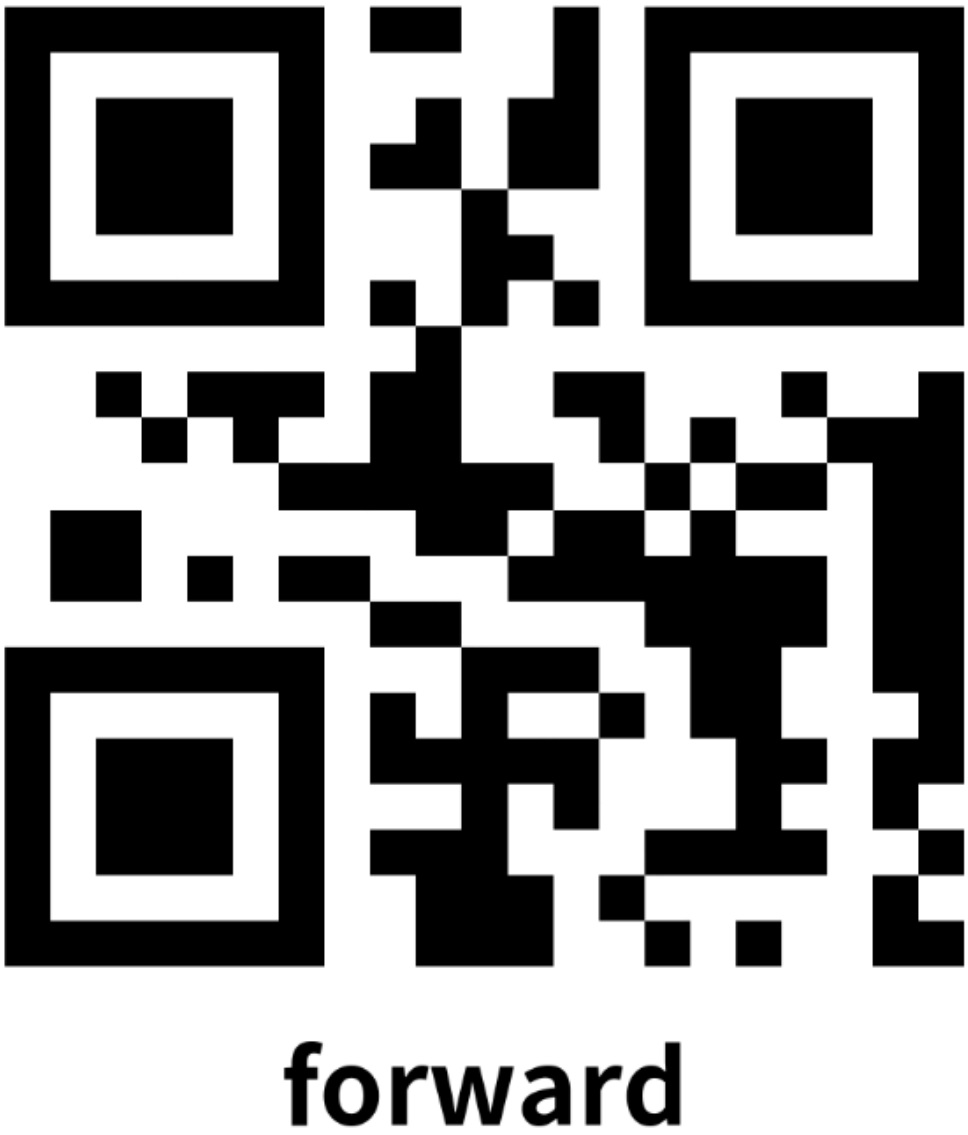
3. Node Description

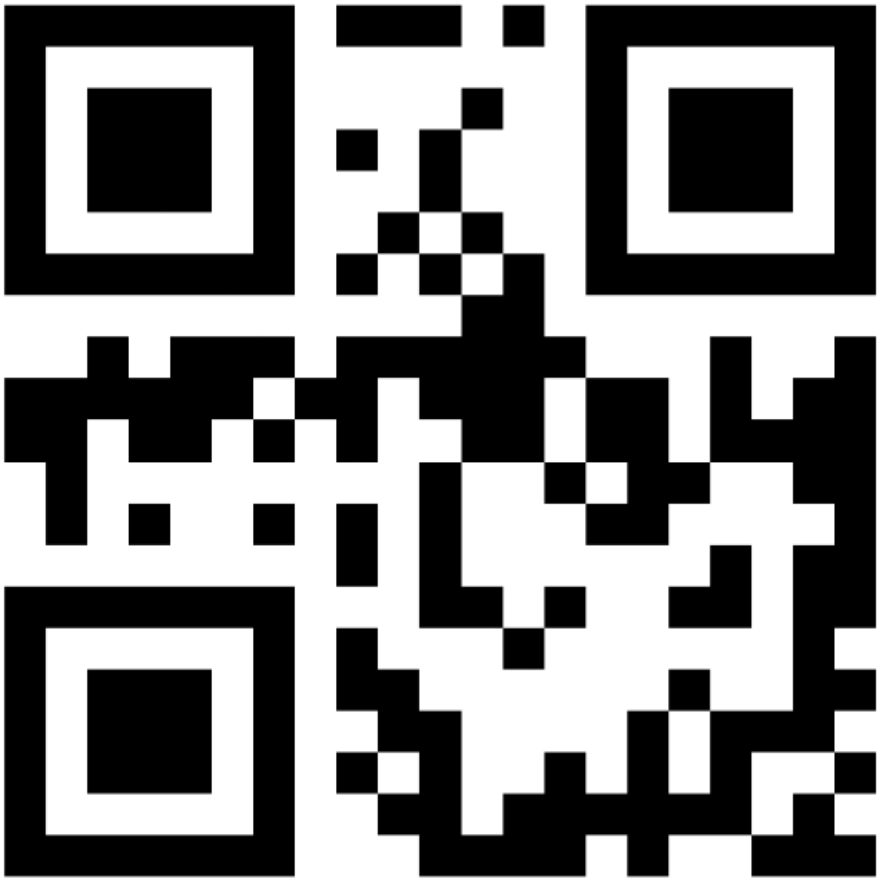
Node	Package	Description
`qr_control_ptz_node`	`microros_v2_ptz_visual_control_pkg`	PTZ QR code recognition + control, dual timers + pan-tilt initialization

6. Frequently Asked Questions

Problem	Cause	Solution
QR code not recognized	Insufficient light or blurred QR code	Ensure the QR code is clear and the lighting is even
Scanned but no action	`auto_start_control=false`	Press the Space key to enable control
Car runs wildly after losing the code	`qr_command_hold_timeout_sec` too large	Reduce the hold timeout (e.g., 0.5 s)
pyzbar unavailable	Missing libzbar0	Install `sudo apt install libzbar0` or use the opencv backend
Abnormal pan-tilt angle	Servo not initialized correctly	Check the PTZ power supply and servo connection

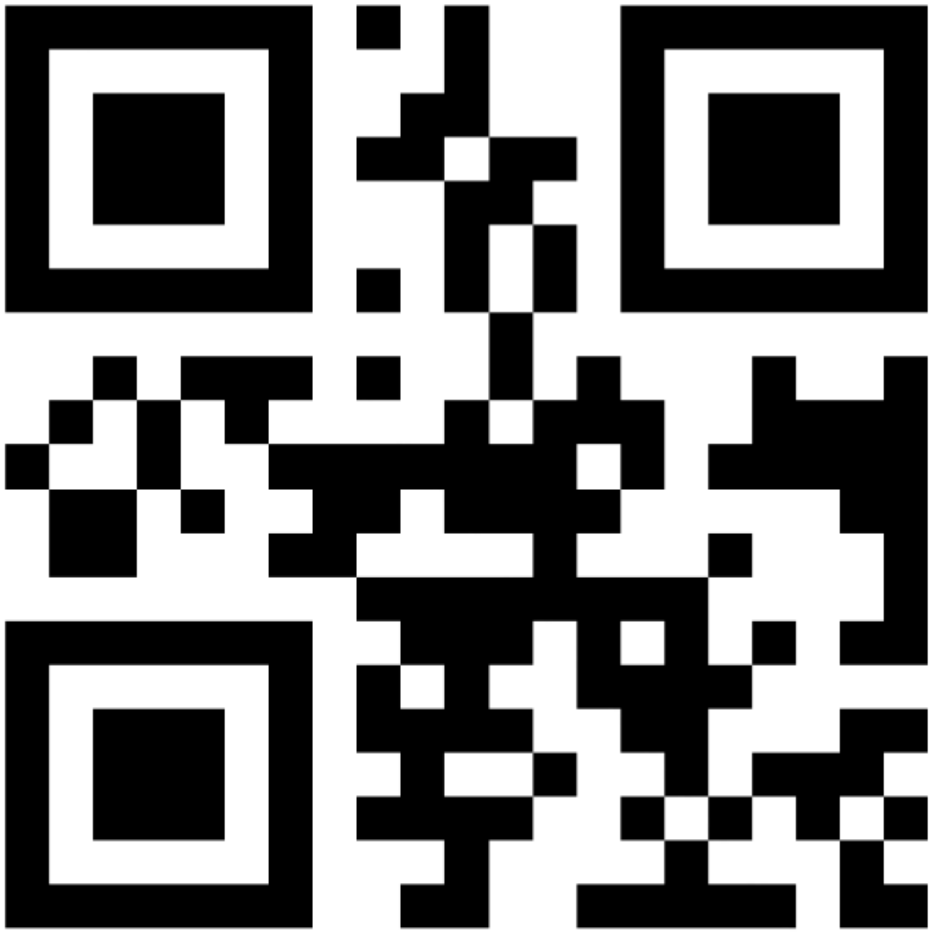
QR codes:



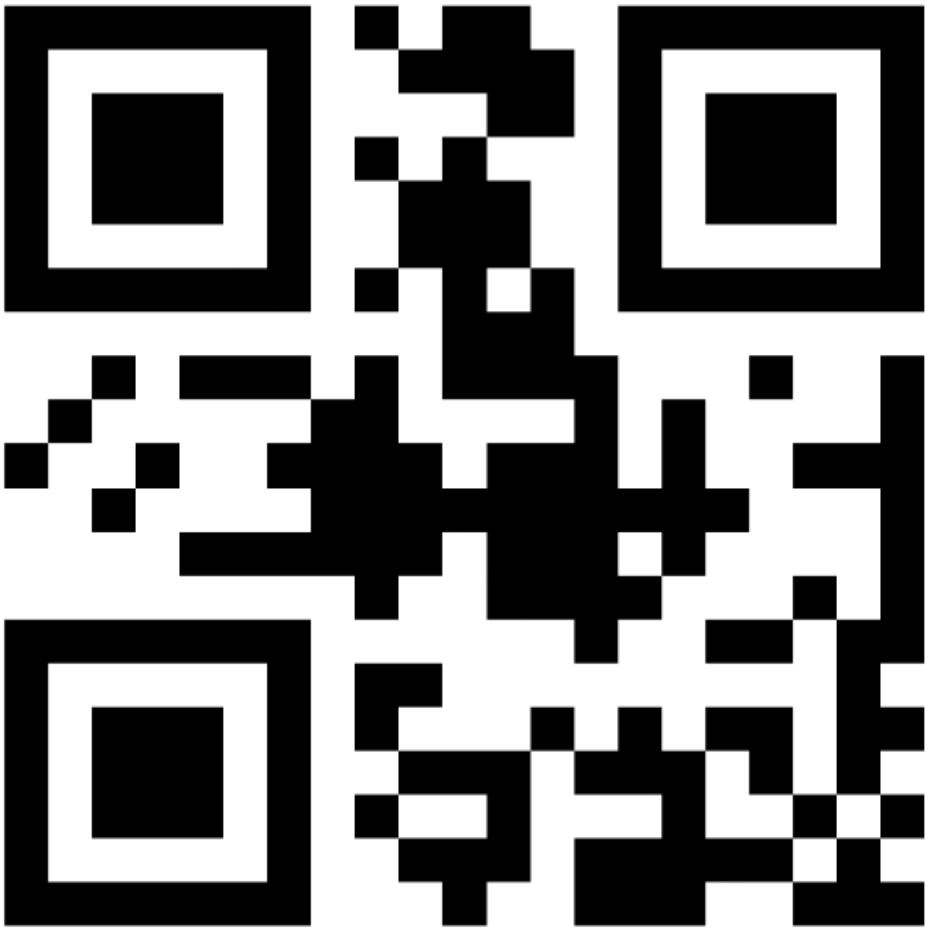


back

left

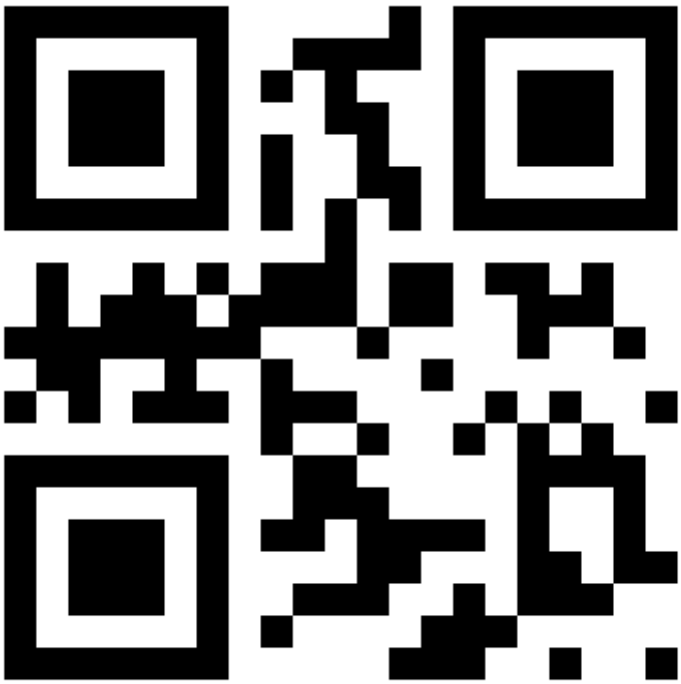


right



stop

turnleft



turnright